EICTA First Project Delivery – ER and SQL

Group Number: 4

Academic Year 2025–2026

Authors:

Annalaura Catalfo — Bocconi: 3200742 — PoliMI: 11182469
Guendalina Nardi - Bocconi: 3376114 — PoliMI: 11179726
Giacomo Nigro — Bocconi: 3186734 — PoliMI: 11182213
Sofia Varotto — Bocconi: 3221388 — PoliMI: 11182433

Contents

1 Introduction **1**
1.1 Provided Text 1
1.2 Text Analysis 2

2 ER Model **3**
2.1 The ER Diagram 3
2.2 Entities (identifiers, attributes, local assumptions) 4
2.2.1 Relationships with attributes 7
2.3 ISA Hierarchies (marked "T,E" on the diagram 7
2.4 Relationships (cardinalities) 8
2.5 Important Reasoning 9

3 Logical Model **10**
3.0.1 From ER to Logical Model 10
3.0.2 ISA Hierarchies 11
3.0.3 Relations and Keys 11
3.0.4 Assumption : Sell / Buy relations (transaction-level modeling) 18

4 SQL - Tables Creation **19**

5 SQL - Data Population **27**

6 SQL - Queries **47**
6.1 Introduction 47
6.2 Query 1: WHERE 47
6.2.1 **ES.1:** *Find all tickets above 100€* 48
6.2.2 **ES.2:** *Find all festivals happening in 2027* 49
6.3 Query 2: WHERE, LIMIT, LIKE 49
6.3.1 **ES.1:** *First 10 customers with email containing "gmail"* 49
6.3.2 **ES.2:** *Italian artists whose stage name starts with 'M'* 50
6.3.3 **ES.3:** *3 cheapest VIP tickets* 51
6.4 Query 3: WHERE, IN, Nested Query 51
6.4.1 **ES.1:** *Customers who bought tickets for "Roma Indie Fest"*

. 51
6.4.2 **ES.2:** *VIP tickets from festivals held in 2028*
. 52
6.4.3 **ES.3:** *Customers who bought both Food and Drinks, with product names*
. 53
6.5 Query 4: GROUP BY, 1 JOIN, AS 53
6.5.1 **ES.1:** *Average ticket price per festival*
. 54
6.5.2 **ES.2:** *Number of stands per festival*
. 54
6.5.3 **ES.3:** *Number of performances per stage*
. 55
6.6 Query 5: WHERE, GROUP BY 57
6.6.1 **ES.1:** *Count food products costing more than 7€*
. 57
6.6.2 **ES.2:** *Number of artists per genre (Italian only)*
. 57
6.6.3 **ES.3:** *Count tickets by type for festivals in 2026*
. 58
6.7 Query 6: GROUP BY, HAVING, AS 59
6.7.1 **ES.1:** *Festivals with more than 2 stands*
. 59
6.7.2 **ES.2:** *Artists with more than 3 performances in ascending order*
. 59
6.7.3 **ES.3:** *Drink types with average price > 4€*
. 60
6.8 Query 7: WHERE, GROUP BY, HAVING, AS 61
6.8.1 **ES.1:** *Festivals in 2027 with average ticket price > 70€*
. 61
6.8.2 **ES.2:** *Customers who bought more than 2 drinks*
. 62
6.9 Query 8: WHERE, Nested Query, GROUP BY 62
6.9.1 **ES.1:** *Customers who bought tickets with an average price above the global average*
. 62
6.9.2 **ES.2:** *Most popular musical genres among festivals held after 2026*
. 63
6.10 Query 9: WHERE, GROUP BY, HAVING, 1 JOIN 64
6.10.1 **ES.1:** *Top 5 festivals with the highest average ticket price*

. 64
6.10.2 **ES.2:** *Artists performing in more than one festival*
. 65
6.11 Query 10:WHERE, GROUP BY, HAVING, 2 JOINs 66
6.11.1 **ES.1:** *Festivals where customers bought the most merchandise items*
. 66
6.11.2 **ES.2:** *Artist (solo and band musician) with highest average ticket price (with regular or VIP ticket pass) across multiple performances*
. 67
6.12 Final Thoughts 68

7 Conclusion **69**

Chapter 1

Introduction

1.1 Provided Text

This chapter introduces the project assignment for the first delivery of the Enterprise ICT Architectures course. It presents the official problem statement provided by the instructor and outlines the scenario on which the database design is based. The following text describes the case study and serves as the foundation for the subsequent phases.

"The Euphoric Events company asked you to design a database for the management of their music festivals. A festival is characterized by a name, a location, a start date, and an end date. Each festival is divided into multiple stages, characterized by an identification code, a name, a maximum capacity, and a description about its location within the festival. At each stage, different musical performances can be held. A performance has a unique identifier, a title, a genre (e.g., rock, jazz, electronic), and start/end times. Performances are given by one or more artists, which may be solo musicians or bands (i.e., at least two musicians). The system should store for each artist their stage name, country, and main musical genre. In addition to this, musicians should provide their full name, nationality and main instrument, whereas bands must register their formation year. Alongside music, festivals can host several stands where people can buy food, drinks and merchandise. Each stand has a name, a sponsor, and can sell either of the three aforementioned products. A product is described by a name and a price; furthermore, food and drinks have an expiration date, and only food items have a type (e.g., vegetarian). To enter a festival, people must buy tickets, which have a type (e.g., regular, day pass, VIP), a price, and a validity period. To buy tickets, a person must state their full name, e-mail address, and phone number. A person can buy multiple tickets, and they can also purchase shop items during the festival. Whenever a purchase happens, the system should store the date and the payment method used."

1.2 Text Analysis

Starting from the real scenario described in the introduction, we translated the context of music festivals into a conceptual model that faithfully represents the entities, relationships, and constraints of the domain. The goal was to build a coherent system that clearly describes all the organizational and commercial aspects of the festival, from performances and artists to the products sold and the tickets purchased by attendees. After carefully analyzing the text, we identified the main entities such as Festival, Stage, Performance, Artist, Stand, Product, Ticket, and Customer, linking them through logical relationships that reflect the real functioning of the event: a festival includes multiple stages, each stage hosts several performances, and each performance may involve one or more artists. At the same time, people purchase tickets and products at stands, generating transactional data that was modeled accordingly. During the design, some ISA generalizations were also identified, such as between Artist and its subclasses SoloMusician and Band, and between Product and its subclasses Food, Drink, and Merchandise. Artificial primary keys were also introduced to avoid ambiguity and ensure consistency among tables. From the ER schema, we then derived the relational logical schema, defining tables, integrity constraints, and the necessary links to build a coherent and complete database. In this way, the initial textual description was transformed into a clear logical structure where every relationship among festivals, performances, artists, stands, and products is precisely represented in accordance with the domain rules.

Chapter 2

ER Model

2.1 The ER Diagram

Building upon the text analysis presented in Chapter 1, this chapter introduces the conceptual representation of the Euphoric Events database through the Entity–Relationship (ER) Model. The ER diagram visually summarizes the entities, their attributes, primary keys, and the relationships identified in the problem statement. It serves as the foundation for the subsequent logical and relational models, ensuring a consistent and normalized structure for the system.

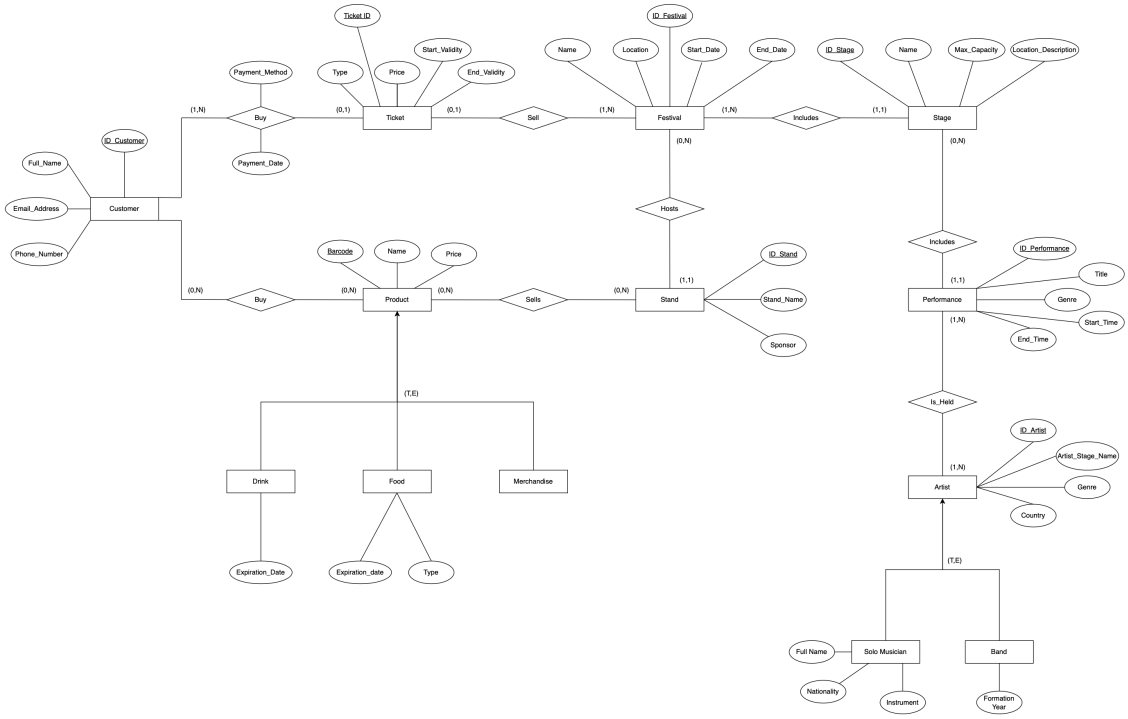

Figure 2.1: ER Model representing entities, attributes, primary and foreign keys, and relationships for the Euphoric Events database. ISA hierarchies are marked as (T,E) to indicate Total and Exclusive specialization.

The following sections describe in detail each component of the ER model — including entities, attributes, relationships, and the design assumptions made during the modeling process.

2.2 Entities (identifiers, attributes, local assumptions)

1 - Festival

PRIMARY KEY (identifier): `ID_Festival`

Attributes: `Name`, `Location`, `Start_Date`, `End_Date`

Assumption: *we introduce an artificial ID because the same festival name can appear in different cities/years (itinerant editions).*

2 - Stage

PRIMARY KEY (identifier): `ID_Stage` *(given by the text as "identification code")*

Attributes: `Name`, `Max_Capacity`, `Location_Description`

Assumptions: ***`Max_Capacity`*** *is a positive quantity;* ***`Location_Description`*** *is an internal description of where the stage sits within the festival area.*

3 - Performance

PRIMARY KEY (identifier): `ID_Performance` *("unique identifier" in the text)*

Attributes: `Title`, `Genre`, `Start_Time`, `End_Time`

Assumption: *performances have a well-defined interval; semantically,* ***`End_Time`*** *must be after* ***`Start_Time`*** *(to be checked later at implementation level).*

4 - Artist (superclass–ISA (T,E) Relation)

PRIMARY KEY (identifier): `ID_Artist`

Attributes: `Artist_Stage_Name` (artistic alias), `Country`, `Genre`

Assumption: *we add an ID because the text does not guarantee uniqueness of the artistic alias.*

Clarification: `Artist_Stage_Name` is *not related* to the entity `Stage`; it is the performer's public alias.

5 - Solo_Musician (ISA child of Artist)

PRIMARY KEY (identifier): *inherits* `ID_Artist` *from Artist*

Attributes: `Full_Name`, `Nationality`, `Instrument`

ISA type: Total, Exclusive

Assumption: *a performer who is a solo artist appears here and not in* ***`Band`****.*

6 - Band (ISA child of Artist)

PRIMARY KEY (identifier): *inherits* `ID_Artist` *from Artist*

Attributes: `Formation_Year`

ISA type: Total, Exclusive

Clarification: *"at least two musicians"* is a conceptual definition of a band; the database is *not* required to record individual members. This information is *descriptive, not relational*; no composition relationship is modeled.

7 - Stand

PRIMARY KEY (identifier): `ID_Stand`

Attributes: `Stand_Name`, `Sponsor`

Assumptions: *we use a surrogate ID to avoid ambiguity, since stands with the same name can participate in different festivals. The* ***`ID_Stand`*** *is assigned at the moment a stand is registered for a specific festival, in the same way a student ID is assigned when a student enrolls at a university. This means that the same stand (or different stands with the same name) can appear in multiple festivals, but each participation receives a different unique* ***`ID_Stand`****, ensuring a clear identification for every festival edition.*

8 - Product (Superclass – ISA (T,E) Relation)

PRIMARY KEY (identifier): `Barcode`

Attributes: `Name`, `Price`

Assumption: *barcodes are globally unique identifiers for a product type. The same item (e.g., Coca-Cola Zero 330ml can) carries the same barcode everywhere, across stands and festivals.* ***`Name`*** *alone is not unique (different products can share similar names), so we rely on* ***`Barcode`*** *to unambiguously identify the product independent of where it is sold.*

9 - Food (ISA child of Product)

PRIMARY KEY (identifier): *inherits* `Barcode` *from Product*

Attributes: `Expiration_Date`, `Type` (e.g., vegetarian)

ISA type: Total, Exclusive

10 - Drink (ISA child of Product)

PRIMARY KEY (identifier): *inherits* `Barcode` *from Product*

Attributes: `Expiration_Date`

ISA type: Total, Exclusive

11 - Merchandise (ISA child of Product)

PRIMARY KEY (identifier): *inherits* `Barcode` *from Product*

Attributes: *(none additional)*

ISA type: Total, Exclusive

12 - Ticket

PRIMARY KEY (identifier): `Ticket_ID` *(artificial primary key, we introduced it because multiple tickets can share type/price/period)*

Attributes: `Type` (regular | day pass | VIP), `Price`, `Start_Validity`, `End_Validity`

Assumption: *the validity period was split into two attributes (`Start_Validity`, `End_Validity`) to facilitate the creation of the SQL tables and to clearly represent the time range in which each ticket is valid.*

13 - Customer

PRIMARY KEY (identifier): `ID_Customer` *(artificial primary key)*

Attributes: `Full_Name`, `Email_Address`, `Phone_Number`

Assumption: *an artificial ID is preferred over e-mail/phone as identifier; e-mail is still expected to be unique in practice, but it would be more computationally expensive.*

2.2.1 Relationships with attributes

1 - Buy (associative entity between Customer and Ticket)

PRIMARY KEY (identifier): composite (`ID_Customer`, `Ticket_ID`)

Attributes: `Payment_Date`, `Payment_Method`

Assumption: *the payment attributes belong to the* ***`Buy`*** *relationship (they describe the transaction, not the ticket or the customer).*

2 - Sell (associative entity between Stand and Product)

PRIMARY KEY (identifier): `ID_Sell`

Attributes: *(none besides identifiers)*

Assumption: ***`Barcode`*** *identifies the product type globally (e.g., "Coca-Cola Zero"). To allow a stand to register multiple sale entries for the same product (and to avoid conflicts the composite key (****`ID_Stand`****,* ***`Barcode`****) would impose), we introduce a surrogate identifier* ***`ID_Sell`*** *that uniquely identifies each sell record (sale entry) between a stand and a product.*

3 - Buy (associative entity between Customer and Product)

PRIMARY KEY (identifier): `ID_Buy`

Assumption: *same reasoning as* ***`Sell_Product`****, we use a surrogate key to uniquely identify each purchase record, allowing the same customer to buy the same product multiple times without key conflicts.*

2.3 ISA Hierarchies (marked "T,E" on the diagram

)

- **Artist** → {`Solo_Musician`, `Band`} = Total, Exclusive (T,E) — each `Artist` has to be either a `Solo_Musician` or a `Band` but not both.
- **Product** → {`Food`, `Drink`, `Merchandise`} = Total, Exclusive (T,E) — each `Product` is either a `Food`, `Drink` or `Merchandise` but not more than one.

2.4 Relationships (cardinalities)

Notation: we list one reading direction with `(min,max)` and then the inverse.

R1) Festival ↔ Includes ↔ Stage

- **Festival → Stage**: (1,N) — each festival is divided into multiple stages (has at least one stage).
- **Stage → Festival**: (1,1) — each stage belongs to exactly one festival, and we assume it cannot be reused (existence-dependent).

R2) Stage ↔ Includes ↔ Performance

- **Stage → Performance**: (1,N) — a stage hosts at least one performance.
- **Performance → Stage**: (1,1) — each performance takes place on one stage.

R3) Performance ↔ Is_Held_By ↔ Artist

- **Performance → Artist**: (1,N) — a performance is given by one or more artists.
- **Artist → Performance**: (1,N) — an artist can participate in one or many performances.

R4) Festival ↔ Hosts ↔ Stand

- **Festival → Stand**: (0,N) — a festival may host none or many stands.
- **Stand → Festival**: (1,1) — every `ID_Stand` operates within exactly one festival; as previously explained, the same stand (or different stands with the same name) can appear in multiple festivals, but each participation receives a different unique `ID_Stand`, ensuring clear identification for every festival edition.

R5) Stand ↔ Sells ↔ Product

- **Stand → Product**: (0,N) — a stand can sell zero or many products.
- **Product → Stand**: (0,N) — a product can be sold at multiple stands.

R6) Customer ↔ Buys ↔ Product

- **Customer → Product**: (0,N) — a customer can buy zero or many products (including repeated purchases of the same product).
- **Product → Customer**: (0,N) — a product can be bought by zero or many customers.

R7) Festival ↔ Sells ↔ Ticket

- **Festival → Ticket**: (1,N) — each festival sells at least one ticket, otherwise it wouldn't exist.
- **Ticket → Festival**: (0,1) — each ticket refers to exactly one festival. *Tickets exist beside the purchase and they can remain unsold.*

R8) Customer ↔ Buys ↔ Ticket

- **Customer → Ticket**: (1,N) — to be considered a customer you must buy at least one ticket; a customer can buy multiple tickets.
- **Ticket → Customer**: (0,1) — a ticket may be unsold (0) or bought once (1).

2.5 Important Reasoning

During the modeling phase, several design choices were made to ensure that the database structure remains consistent, normalized, and adaptable. Artificial identifiers were introduced wherever natural attributes could not guarantee uniqueness or long-term stability — for example, `ID_Stand` for stands that may appear in different festivals and `ID_Customer` for customers whose e-mail could change.

ISA hierarchies were modeled as Total and Exclusive (T,E) to avoid attribute redundancy and maintain clear distinctions between subtypes, such as `Solo_Musician` and `Band` under `Artist`, and `Food`, `Drink`, and `Merchandise` under `Product`.

Associative entities like `Performance_Artist`, `Sell_Product`, and `Buy_Product` were introduced to manage many-to-many relationships while preserving referential integrity. These modeling choices ensure data accuracy and simplify the transition from the conceptual to the logical level.

The ER Model developed for *Euphoric Events* provides a comprehensive conceptual representation of the system, integrating all entities, attributes, relationships, and constraints derived from the problem statement. It ensures consistency and data integrity across all festival-related components — from performances and artists to sales and ticket management. This conceptual foundation will serve as the basis for the Logical Model discussed in the next chapter, where each entity and relationship will be translated into relational tables with defined primary and foreign keys.

Chapter 3

Logical Model

Every previous assumption is still valid throughout these developments.

3.0.1 From ER to Logical Model

The logical model was derived from the conceptual ER diagram following the standard transformation methodology:

1. **Create one relation for each entity.**
   Each entity becomes a logical relation containing all its attributes and its primary key (PK).

2. **Transform ISA hierarchies according to their type.**
   The model contains two *Total and Exclusive (T,E)* hierarchies (`Artist` → `Solo_Musician/Band` and `Product` → `Food/Drink/Merchandise`).
   In this case, only the child entities are transformed, and each inherits all the attributes and the primary key from the parent.

3. **Assign foreign keys (FKs) and relationship attributes following cardinalities:**

   - In *One-to-One* relationships, the designer decides where to place the FK (design choice).
   - In *One-to-Many* relationships, the FK is placed in the entity on the "many" side.
   - In *Many-to-Many (M:N)* relationships, a bridge table is created.
     This table contains the *Primary Keys (PKs)* of both participating entities, which together form a **composite primary key** for the bridge table. Each of these two attributes not only forms part of the composite PK but also serves as a *Foreign Key (FK)* referencing the corresponding parent entity.

This guarantees **referential integrity** and ensures that every relationship instance stored in the bridge table corresponds to valid tuples in both related entities.

3.0.2 ISA Hierarchies

Artist → (Solo_Musician, Band)

This hierarchy is **Total and Exclusive (T,E)**. Every `Artist` must be either a `Solo_Musician` or a `Band`, never both.

The superclass `Artist` is not implemented as a separate table; only the child entities are transformed, inheriting all attributes from the parent (`Country`, `Genre`, `Artist_Stage_Name`) along with their specific ones.

Product → (Food, Drink, Merchandise)

This hierarchy is also **Total and Exclusive (T,E)**. Each product is exactly one of the three types. The superclass `Product` is not transformed; instead, three separate relations (`Food`, `Drink`, `Merchandise`) are created, each inheriting all common attributes from `Product` (`Name`, `Price`) plus their specific ones.

3.0.3 Relations and Keys

→ Festival

`Festival (ID_Festival, Name, Location, Start_Date, End_Date)`

- **PK:** `ID_Festival`
- No FKs, as it is the top-level entity.

→ Stage

`Stage (ID_Stage, Name, Max_Capacity, Location_Description, ID_Festival)`

- **PK:** `ID_Stage`

- **FK:** ID_Festival → Festival(ID_Festival)

Derived from the 1:N relationship Festival-Stage. Each festival can include multiple stages, and each stage belongs to one festival.

→ Performance

Performance (ID_Performance, Title, Genre, Start_Time, End_Time, ID_Stage)

- **PK:** ID_Performance
- **FK:** ID_Stage → Stage(ID_Stage)

Derived from the 1:N relationship Stage-Performance. Each performance occurs on one stage but a stage can host many performances.

→ Artist_Solo_Musician

Artist_Solo_Musician (ID_Solo_Musician, Country, Genre, Artist_Stage_Name, Full_Name, Nationality, Instrument)

- **PK:** ID_Solo_Musician

Derived from the T,E hierarchy Artist → Solo_Musician/Band. Inherits all attributes from Artist plus those specific to solo musicians.

→ Artist_Band

Artist_Band (ID_Band, Country, Genre, Artist_Stage_Name, Formation_Year, Band_Name)

- **PK:** ID_Band

Derived from the same hierarchy. Inherits all common attributes from `Artist` and adds band-specific ones.

→ Performance_Solo_Musician

`Performance_Solo_Musician (ID_Performance, ID_Solo_Musician)`

- **PK:** (ID_Performance, ID_Solo_Musician)
- **FKs:**
  - ID_Performance → `Performance(ID_Performance)`
  - ID_Solo_Musician → `Artist_Solo_Musician(ID_Solo_Musician)`
- Implements the M:N relationship between `Performance` and `Solo_Musician`.

→ Performance_Band

`Performance_Band (ID_Performance, ID_Band)`

- **PK:** (ID_Performance, ID_Band)
- **FKs:**
  - ID_Performance → `Performance(ID_Performance)`
  - ID_Band → `Artist_Band(ID_Band)`
- Implements the M:N relationship between `Performance` and `Band`.

Design choice for the performance of the artists: We used two bridge tables (`Performance_Solo_Musician`, `Performance_Band`) instead of one with nullable columns to avoid `NULL` values in keys, maintain referential integrity, and ensure semantic clarity.

→ **Stand**

`Stand (ID_Stand, Stand_Name, Sponsor, ID_Festival)`

- **PK:** `ID_Stand`
- **FK:** `ID_Festival` → `Festival(ID_Festival)`

→ **Ticket**

`Ticket (Ticket_ID, Type, Price, Start_Validity, End_Validity, ID_Festival)`

- **PK:** `Ticket_ID`
- **FK:** `ID_Festival` → `Festival(ID_Festival)`

→ **Customers**

`Customers (ID_Customer, Full_Name, Email_Address, Phone_Number)`

- **PK:** `ID_Customer`

→ **Ticket_Payment**

`Ticket_Payment (Ticket_ID, ID_Customer, Date, Payment_Method)`

- **PK:** `Ticket_ID`
- **FKs:**

- Ticket_ID → Tickets(Ticket_ID)
- ID_Customer → Customers(ID_Customer)

→ Product_Drink

Product_Drink (Barcode_Drink, Name, Price, Expiration_Date)

- **PK:** Barcode_Drink

Derived from T,E hierarchy Product → Drink. Contains both inherited and specific attributes.

→ Product_Food

Product_Food (Barcode_Food, Name, Price, Expiration_Date, Type)

- **PK:** Barcode_Food

Derived from T,E hierarchy Product → Food. Contains both inherited and specific attributes.

→ Product_Merchandise

Product_Merchandise (Barcode_Merchandise, Name, Price)

- **PK:** Barcode_Merchandise

Derived from T,E hierarchy Product → Merchandise. No additional attributes.

Sell relations (Stand × Product subtypes)

→ Sell_Food

`Sell_Food (ID_Sell, ID_Stand, Barcode_Food)`

- **PK:** `ID_Sell`
- **FKs:**
  - `ID_Stand` → `Stand(ID_Stand)`
  - `Barcode_Food` → `Food(Barcode_Food)`

→ Sell_Drink

`Sell_Drink (ID_Sell, ID_Stand, Barcode_Drink)`

- **PK:** `ID_Sell`
- **FKs:**
  - `ID_Stand` → `Stand(ID_Stand)`
  - `Barcode_Drink` → `Drink(Barcode_Drink)`

→ Sell_Merchandise

`Sell_Merchandise (ID_Sell, ID_Stand, Barcode_Merchandise)`

- **PK:** `ID_Sell`
- **FKs:**
  - `ID_Stand` → `Stand(ID_Stand)`
  - `Barcode_Merchandise` → `Merchandise(Barcode_Merchandise)`

Buy relations (Customer × Product subtypes)

→ Buy_Food

`Buy_Food (ID_Buy, ID_Customer, Barcode_Food)`

- **PK:** ID_Buy
- **FKs:**
  - ID_Customer → Customers(ID_Customer)
  - Barcode_Food → Food(Barcode_Food)

→ Buy_Drink

`Buy_Drink (ID_Buy, ID_Customer, Barcode_Drink)`

- **PK:** ID_Buy
- **FKs:**
  - ID_Customer → Customers(ID_Customer)
  - Barcode_Drink → Drink(Barcode_Drink)

→ Buy_Merchandise

`Buy_Merchandise (ID_Buy, ID_Customer, Barcode_Merchandise)`

- **PK:** ID_Buy
- **FKs:**
  - ID_Customer → Customers(ID_Customer)
  - Barcode_Merchandise → Merchandise(Barcode_Merchandise)

3.0.4 Assumption : Sell / Buy relations (transaction-level modeling)

Scope and rationale

We model each sale/purchase as a distinct transaction. To identify transactions unambiguously, we introduce a surrogate key (`ID_Sell` / `ID_Buy`) rather than relying on a composite key such as (`ID_Stand`, `Barcode`) or (`ID_Customer`, `Barcode`).

This avoids ambiguity when the same stand sells the same product multiple times (or the same customer buys the same product multiple times) and makes it easy to attach future transactional attributes (e.g., timestamp, quantity, discount, unit price, etc.).

Why a surrogate `ID_Sell`?

- The pair (`ID_Stand`, `Barcode`) is not unique over time (a stand can sell the same product many times).
- Using a surrogate key uniquely identifies each sale event regardless of product category.

Why a surrogate `ID_Buy`?

- The pair (`ID_Customer`, `Barcode`) does not uniquely identify purchases (a customer can buy the same item many times).
- `ID_Buy` gives each purchase its own identity, enabling clean auditing and easy attachment of transactional attributes.

In both cases (`Sell`, `Buy`), by using these two artificial PKs, each transaction links directly and unambiguously to the correct product subtype:

- No `NULL`s appear in foreign keys or primary keys.
- Referential integrity is guaranteed (each barcode always references its correct subtype).
- Constraints and indexes remain simple and efficient.
- Each table is compact, homogeneous, and semantically clear.

Chapter 4

SQL - Tables Creation

The following section defines the foundational structure of a relational database designed to organize and manage a complex event-based system. It establishes key entities, their attributes, and the relationships that ensure data integrity across various interconnected components. The schema enforces referential consistency through foreign keys and includes constraints to maintain logical and temporal coherence among records. Overall, it provides a robust framework for storing, querying, and maintaining information about multiple interrelated activities and resources.

```
PRAGMA foreign_keys = ON;


-------------------------------------------------------------
1) FESTIVAL
-------------------------------------------------------------

CREATE TABLE Festival (
  ID_Festival INTEGER PRIMARY KEY AUTOINCREMENT,
  Name VARCHAR(100) NOT NULL,
  Location VARCHAR(50) NOT NULL,
  Start_Date DATE NOT NULL,
  End_Date DATE NOT NULL,
  CHECK (End_Date >= Start_Date)
);
```

This table defines the general structure for managing event instances. It includes identifying information, location, and temporal boundaries that describe the duration of each event. The integrity constraint ensures that the end date cannot precede the start date.

```
------------------------------------------------------------
2) STAGE
------------------------------------------------------------
CREATE TABLE Stage (
  ID_Stage TEXT PRIMARY KEY,
  Name VARCHAR(100) NOT NULL,
  Max_Capacity INT NOT NULL CHECK (Max_Capacity > 0),
  Location_Description VARCHAR(250),
  ID_Festival INT NOT NULL,
  FOREIGN KEY (ID_Festival) REFERENCES Festival(ID_Festival)
    ON DELETE CASCADE
);
```

This table stores information about the various areas or locations within an event where activities occur. It records the capacity and spatial description of each location and maintains a link to the main event through a foreign key relationship. The capacity constraint prevents the registration of invalid or negative values.

```
------------------------------------------------------------
3) PERFORMANCE
------------------------------------------------------------

CREATE TABLE Performance (
  ID_Performance  TEXT PRIMARY KEY,
  Title VARCHAR(160) NOT NULL,
  Genre VARCHAR(50) NOT NULL,
  Start_Time DATETIME NOT NULL,
  End_Time DATETIME NOT NULL,
  ID_Stage TEXT NOT NULL,
  CHECK (End_Time > Start_Time),
  FOREIGN KEY (ID_Stage) REFERENCES Stage(ID_Stage)
    ON DELETE CASCADE
);
```

This table represents scheduled activities or acts taking place during an event. It includes descriptive attributes, time constraints, and a foreign key connection to the specific stage where each activity is performed. The temporal check guarantees that all performances have a logical time order.

```
------------------------------------------------------------
4) ARTISTS
------------------------------------------------------------

CREATE TABLE Artist_Solo_Musician (
  ID_Solo_Musician TEXT PRIMARY KEY,
  Country VARCHAR(30),
  Genre VARCHAR(30),
  Artist_Stage_Name VARCHAR(30),
  Full_Name VARCHAR(30),
  Nationality VARCHAR(30),
  Instrument VARCHAR(30)
);
```

This table stores information about individual performers. It includes attributes describing origin, style, and identifying information, as well as optional details such as instruments played. Each record represents a unique solo artist.

```
CREATE TABLE Band (
  ID_Band TEXT PRIMARY KEY,
  Country VARCHAR(30),
  Genre VARCHAR(30),
  Artist_Stage_Name VARCHAR(30),
  Formation_Year YEAR,
  Band_Name VARCHAR(30)
);
```

This structure defines collective performer entities. It captures relevant descriptive attributes including musical style, year of formation, and identifying names. Each record corresponds to one performing group or ensemble.

```
------------------------------------------------------------
5) PERFORMANCE SOLO MUSICIAN & BAND
------------------------------------------------------------

CREATE TABLE Performance_Solo_Musician (
  ID_Performance TEXT,
  ID_Solo_Musician TEXT,
  PRIMARY KEY (ID_Performance, ID_Solo_Musician),
  FOREIGN KEY (ID_Performance) REFERENCES Performance(ID_Performance),
  FOREIGN KEY (ID_Solo_Musician) REFERENCES Artist_Solo_Musician(ID_Solo_Musician)
    ON DELETE CASCADE
);
```

This associative table connects solo artists to the activities or sessions in which they participate. It establishes a many-to-many relationship, allowing each performer to

appear in multiple events and vice versa. Referential integrity ensures that records remain consistent with existing artist and performance data.

```
CREATE TABLE Performance_Band (
  ID_Performance TEXT,
  ID_Band TEXT,
  PRIMARY KEY (ID_Performance, ID_Band),
  FOREIGN KEY (ID_Performance) REFERENCES Performance(ID_Performance),
  FOREIGN KEY (ID_Band) REFERENCES Band(ID_Band)
    ON DELETE CASCADE
);
```

This table mirrors the previous relationship model but links groups instead of individual performers. It manages connections between collective entities and scheduled sessions, enforcing referential integrity with cascaded deletions.

```
---------------------------------------------------------------
6) STAND
---------------------------------------------------------------

CREATE TABLE Stand (
  ID_Stand TEXT PRIMARY KEY,
  Stand_Name VARCHAR(80) NOT NULL,
  Sponsor VARCHAR(80),
  ID_Festival  INTEGER NOT NULL,
  FOREIGN KEY (ID_Festival) REFERENCES Festival(ID_Festival)
    ON DELETE CASCADE
);
```

This table describes physical or commercial areas present within each event. It includes identifying information and sponsorship details while maintaining a foreign key link to the associated event. Deleting a parent event cascades removal of its dependent stands.

```
---------------------------------------------------------------
7) PRODUCT: DRINK - FOOD - MERCHANDISE
---------------------------------------------------------------

CREATE TABLE Product_Drink (
  Barcode_Drink INT PRIMARY KEY,
  Name VARCHAR(20),
  Price INT,
  Expiration_Date DATE
);
```

This table defines the range of liquid consumables available within the system. It stores identifying codes, descriptive labels, pricing information, and validity periods to ensure proper inventory control.

```
CREATE TABLE Product_Food (
  Barcode_Food INT PRIMARY KEY,
  Name VARCHAR(20),
  Price INT,
  Expiration_Date DATE,
  Type VARCHAR(20)
);
```

This table manages edible items offered in various venues. It contains identification, pricing, and classification details such as food type or dietary category. Expiration tracking supports quality assurance.

```
CREATE TABLE Product_Merchandise (
  Barcode_Merchandise INT PRIMARY KEY,
  Name VARCHAR(20),
  Price INT
);
```

This table lists non-consumable commercial goods. It includes identification and pricing information, supporting the management of retail merchandise associated with events or venues.

```
-------------------------------------------------------------
8) SELL PRODUCT: FOOD - DRINK - MERCHANDISE
-------------------------------------------------------------

CREATE TABLE Sell_Food (
  ID_Sell TEXT PRIMARY KEY,
  ID_Stand TEXT,
  Barcode_Food INT,
  FOREIGN KEY (ID_Stand) REFERENCES Stand(ID_Stand),
  FOREIGN KEY (Barcode_Food) REFERENCES Product_Food(Barcode_Food)
    ON DELETE CASCADE
);
```

This table links points of sale to specific edible items. It records which locations distribute particular food products and enforces relational consistency between stand and product data.

```
CREATE TABLE Sell_Drink (
  ID_Sell TEXT PRIMARY KEY,
  ID_Stand TEXT,
  Barcode_Drink INT,
  FOREIGN KEY (ID_Stand) REFERENCES Stand(ID_Stand),
  FOREIGN KEY (Barcode_Drink) REFERENCES Product_Drink(Barcode_Drink)
    ON DELETE CASCADE
);
```

This structure captures the sale of drink products within various locations. Each record identifies both the sales point and the product offered, ensuring valid foreign key relationships to prevent orphan entries.

```
CREATE TABLE Sell_Merchandise (
  ID_Sell TEXT PRIMARY KEY,
  ID_Stand TEXT,
  Barcode_Merchandise INT,
  FOREIGN KEY (ID_Stand) REFERENCES Stand(ID_Stand),
  FOREIGN KEY (Barcode_Merchandise) REFERENCES Product_Merchandise(Barcode_Merchand
    ON DELETE CASCADE
);
```

This table documents the sale of promotional or retail goods within individual commercial points. It maintains referential links to both merchandise records and their selling stands, allowing traceability and consistency in inventory management.

---

9) CUSTOMER

```
CREATE TABLE Customer (
  ID_Customer TEXT PRIMARY KEY,
  Full_Name VARCHAR(100) NOT NULL,
  Email_Address VARCHAR(100) NOT NULL,
  Phone_Number VARCHAR(20) NOT NULL,
  UNIQUE (Email_Address, Phone_Number)
);
```

This table contains general information about registered participants or clients. It stores unique identifiers and contact details, ensuring no duplicate entries for the same email and phone number. This allows reliable communication and consistent customer management.

```
------------------------------------------------------------
10) TICKET
------------------------------------------------------------

CREATE TABLE Ticket (
  ID_Ticket TEXT PRIMARY KEY,
  Ticket_Type VARCHAR(20) NOT NULL CHECK (Ticket_Type IN ('REGULAR','DAY_PASS','VIP
  Price DECIMAL(8,2) NOT NULL,
  Start_Validity DATE NOT NULL,
  End_Validity DATE NOT NULL,
  ID_Festival INTEGER NOT NULL,
  FOREIGN KEY (ID_Festival) REFERENCES Festival(ID_Festival)
    ON DELETE CASCADE,
  CHECK (Start_Validity <= End_Validity)
);
```

This table defines different types of access passes. Each record includes pricing, validity periods, and an association with a particular event. Integrity constraints ensure that the ticket type is valid and that its time frame follows a logical order.

```
------------------------------------------------------------
11) TICKET PAYMENT
------------------------------------------------------------

CREATE TABLE Ticket_Payment (
  Ticket_ID TEXT NOT NULL,
  ID_Customer INT NOT NULL,
  Payment_Date DATE NOT NULL,
  Payment_Method VARCHAR(30) NOT NULL,
  FOREIGN KEY (Ticket_ID) REFERENCES Ticket(ID_Ticket)
    ON DELETE CASCADE,
  FOREIGN KEY (ID_Customer) REFERENCES Customer(ID_Customer)
    ON DELETE RESTRICT
  PRIMARY KEY (Ticket_ID, ID_Customer)
);
```

This table records payment transactions made by customers for event access. It associates tickets with their buyers and stores payment details, including the method and date. The combined primary key prevents duplication of identical transactions.

```
------------------------------------------------------------
12) BUY PRODUCT: FOOD - DRINK - MERCHANDISE
------------------------------------------------------------

CREATE TABLE Buy_Food (
  ID_Buy TEXT PRIMARY KEY,
  ID_Customer TEXT,
  Barcode_Food INT,
  FOREIGN KEY (ID_Customer) REFERENCES Customer(ID_Customer),
  FOREIGN KEY (Barcode_Food) REFERENCES Product_Food(Barcode_Food)
    ON DELETE CASCADE
);
```

This table logs purchases of edible products. Each record connects a customer to a specific food item and ensures that both customer and product references remain valid. It supports traceability of individual purchases.

```
CREATE TABLE Buy_Drinks (
  ID_Buy TEXT PRIMARY KEY,
  ID_Customer TEXT,
  Barcode_Drink INT,
  FOREIGN KEY (ID_Customer) REFERENCES Customer(ID_Customer),
  FOREIGN KEY (Barcode_Drink) REFERENCES Product_Drink(Barcode_Drink)
    ON DELETE CASCADE
);
```

This table documents beverage transactions. It establishes a relationship between customers and the drink products they acquire. Referential constraints ensure consistency with existing customer and product data.

```
CREATE TABLE Buy_Merchandise (
  ID_Buy TEXT PRIMARY KEY,
  ID_Customer TEXT,
  Barcode_Merchandise INT,
  FOREIGN KEY (ID_Customer) REFERENCES Customer(ID_Customer),
  FOREIGN KEY (Barcode_Merchandise) REFERENCES Product_Merchandise(Barcode_Merchand
    ON DELETE CASCADE
);
```

This table captures the purchase of non-consumable goods by customers. It connects merchandise items to buyers, maintaining referential integrity with the relevant customer and product records. This structure allows for detailed analysis of retail trends and user engagement.

Chapter 5

SQL - Data Population

The *Euphoric Events* database was designed to simulate a realistic yet manageable digital environment representing the organization and operation of music festivals across Italy. It integrates multiple entities describing festivals, artists, performances, customers, tickets, stands, and product sales. The model ensures referential integrity, normalization, and a logical separation of responsibilities across entities, making it suitable for SQL querying and managerial interpretation.

For simplicity, in this report we have shown the first 25 example queries, while the full tables with all the data are already included in the project folder, showing the complete structure of the system.

Table 5.1: The *Festival* entity serves as the central hub of the database, encompassing 25 events located throughout Italy, from Milan to Palermo, each lasting between one and four days.

Name	**Location**	**Start Date**	**End Date**
Milano Soundwave	Milano	2025-06-20	2025-06-22
Roma Indie Fest	Roma	2025-07-10	2025-07-13
Firenze Summer Jam	Firenze	2025-08-05	2025-08-06
Napoli Beats	Napoli	2025-09-01	2025-09-03
Torino Vibes	Torino	2026-06-14	2026-06-15
Bologna Groove Fest	Bologna	2026-07-02	2026-07-04
Verona Rock Nights	Verona	2026-08-10	2026-08-12
Genova Sea Sounds	Genova	2026-09-05	2026-09-05
Bari Music Garden	Bari	2027-06-22	2027-06-23
Palermo Sunset Fest	Palermo	2027-07-13	2027-07-15
Cagliari Wave	Cagliari	2027-08-18	2027-08-20
Padova Indie Days	Padova	2027-09-10	2027-09-11
Lecce Live Energy	Lecce	2028-06-12	2028-06-14
Trieste Electro Sound	Trieste	2028-07-01	2028-07-01
Parma Classic Festival	Parma	2028-07-20	2028-07-22
Perugia Jazz Fusion	Perugia	2028-08-03	2028-08-04
Ancona Pop Nights	Ancona	2028-08-25	2028-08-27
Rimini Beach Beats	Rimini	2028-09-06	2028-09-09
Pisa Sound Arena	Pisa	2027-06-15	2027-06-17
Modena Metal Storm	Modena	2026-07-08	2026-07-08
Reggio Vibes	Reggio Emilia	2025-08-01	2025-08-03
Lucca Music Days	Lucca	2026-09-10	2026-09-12
Aosta Alpine Jam	Aosta	2027-07-14	2027-07-14
Pescara Festival del Mare	Pescara	2027-08-09	2027-08-10
Como Lakeside Fest	Como	2028-06-21	2028-06-23

Table 5.2: Every festival contains a varying number of *Stages* (from one to five, for a total of 50), each defined by a name, maximum capacity, and a specific location within the venue. This hierarchical design allows for multi-stage events and reflects the complexity of large-scale festivals.

ID Stage	Name	Max Capacity	Location Description	ID Festival
S001	Main Stage Milano	20000	Central area of Parco Sempione	1
S002	Sunset Arena Milano	15000	South area, near lake	1
S003	Indie Garden Milano	12000	West entrance section	1
S004	Indie Arena Roma	10000	Villa Ada, west area	2
S005	Electronic Dome Roma	8000	North expo pavilion	2
S006	Rock Park Roma	13000	Villa Borghese main field	2
S007	Acoustic Stage Roma	9000	South garden section	2
S008	Florence Jazz Hall	11000	Campo di Marte zone	3
S009	Acoustic Garden Firenze	8000	Giardini Cascine, southern side	3
S010	Napoli Rock Stage	16000	Port area main stage	4
S011	Napoli Underground Dome	10000	Old metro zone arena	4
S012	Napoli Sunset Area	9000	Seaside east zone	4
S013	Torino Main Square	18000	Piazza Castello central stage	5
S014	Torino Electro Hall	9500	Lingotto Expo space	5
S015	Bologna Groove Stage	9000	Parco Nord central field	6
S016	Funk Garden Bologna	7000	Green north section	6
S017	Verona Rock Arena	16000	Arena di Verona open air	7
S018	Indie Zone Verona	9000	Piazza Bra east section	7
S019	Acoustic Stage Verona	8500	Porta Nuova park	7
S020	Genova Sea Front	11000	Porto Antico area	8
S021	Bari Green Garden	10000	City park east side	9
S022	Bari Acoustic Bay	6000	South park area	9
S023	Palermo Sunset Arena	13000	Mondello beach area	10

ID Stage	Name	Max Capacity	Location Description	ID Festival
S024	Island Stage Palermo	9000	Lungomare north section	10

Table 5.3: The *Performance* table contains 60 entries, each representing a live show held during one of the festivals. Each performance is linked to a specific stage, ensuring the correct association between the show, its location, and the corresponding festival. The dataset covers a wide variety of musical styles and events across different Italian cities.

ID Performance	Title	Genre	Start Time	End Time	ID Stage
P001	Neon Echoes	Electronic	2025-06-20 21:00	2025-06-20 23:30	S001
P002	Dreamline Tour	Pop	2025-06-21 20:00	2025-06-21 23:00	S002
P003	Midnight Frequencies	Rock	2025-06-22 19:00	2025-06-22 22:00	S003
P004	Velvet Horizon	Indie	2025-07-10 19:30	2025-07-10 22:30	S004
P005	Electric Mirage	Electronic	2025-07-11 22:00	2025-07-12 01:00	S005
P006	Golden Pulse	Pop-Rock	2025-07-12 19:00	2025-07-12 22:30	S006
P007	Moonlight Jazz Ensemble	Jazz	2025-08-05 21:00	2025-08-05 23:30	S008
P008	Hidden Stories	Folk	2025-08-06 18:30	2025-08-06 21:30	S009
P009	Chromatic Wave	Electronic	2025-09-01 22:00	2025-09-02 01:30	S011
P010	Steel Strings	Rock	2025-09-02 20:00	2025-09-02 23:00	S012
P011	Digital Bloom	Electronic	2026-06-14 21:00	2026-06-15 00:00	S013
P012	Northern Lights	Pop	2026-06-15 18:00	2026-06-15 21:00	S014
P013	Groove Mechanics	Funk	2026-07-02 20:00	2026-07-02 23:00	S015
P014	The Velvet Flow	Soul	2026-07-03 19:30	2026-07-03 22:30	S016
P015	Crimson Horizon	Rock	2026-08-10 19:00	2026-08-10 22:30	S017
P016	Indigo Nights	Indie	2026-08-11 20:00	2026-08-11 23:00	S018
P017	Ocean Frequencies	Electronic	2026-09-05 21:00	2026-09-05 23:30	S019
P018	Summer Groove	Pop	2027-06-22 18:30	2027-06-22 22:00	S021
P019	Soulful Lines	Jazz	2027-06-23 19:00	2027-06-23 22:00	S022
P020	Dancing Mirage	Pop	2027-07-13 18:00	2027-07-13 22:00	S023
P021	Afterglow	Electronic	2027-08-18 22:00	2027-08-19 01:00	S025
P022	Silent Avenue	Indie	2027-09-10 20:00	2027-09-10 23:00	S027
P023	Symphonic Dawn	Classical	2028-06-12 21:00	2028-06-12 23:00	S029

ID Performance	Title	Genre	Start Time	End Time	ID Stage
P024	Digital Rain	Electronic	2028-07-01 19:00	2028-07-01 22:00	S031

Artists:

Solo Musicians

The *Solo Musician* table contains 50 artists. The dataset represents a wide range of both Italian and international artists, covering diverse musical styles.

Band

The *band* table includes 50 groups. the database represents different musical genres and time periods within the pop, rock, electronic, and jazz scenes.

Table 5.4: Solo Musicians

ID Solo Musician	Country	Genre	Artist Stage Name	Full Name	Nationality	Instrument
SM01	Italy	Pop	Elisa	Elisa Toffoli	Italian	vocals
SM02	Italy	Pop	Tiziano Ferro	Tiziano Ferro	Italian	vocals
SM03	Italy	Jazz	Paolo Fresu	Paolo Fresu	Italian	trumpet
SM04	Italy	Rock	Zucchero	Adelmo Fornaciari	Italian	guitar
SM05	Italy	Classical	Ludovico Einaudi	Ludovico Einaudi	Italian	piano
SM06	Italy	Pop	Marco Mengoni	Marco Mengoni	Italian	vocals
SM07	Italy	Folk	Francesco De Gregori	Francesco De Gregori	Italian	guitar
SM08	Italy	Indie	Levante	Claudia Lagona	Italian	vocals
SM09	Italy	Pop	Laura Pausini	Laura Pausini	Italian	vocals
SM10	Italy	Soul	Mara Sattei	Sara Mattei	Italian	vocals
SM11	UK	Pop	Adele	Adele Adkins	British	vocals
SM12	UK	Rock	Ed Sheeran	Edward Sheeran	British	guitar
SM13	UK	Jazz	Jamie Cullum	Jamie Cullum	British	piano
SM14	UK	Indie	Arlo Parks	Anaïs Parks	British	vocals
SM15	USA	Rock	Jack White	John White	American	guitar
SM16	USA	Pop	Billie Eilish	Billie Eilish	American	vocals
SM17	CAN	R&B	The Weeknd	Abel Tesfaye	Canadian	vocals
SM18	France	Electronic	David Guetta	David Guetta	French	other
SM19	Germany	Electronic	Robin Schulz	Robin Schulz	German	other
SM20	Spain	Pop	Rosalía	Rosalía Vila Tobella	Spanish	vocals
SM21	Belgium	Electronic	Stromae	Paul Van Haver	Belgian	vocals
SM22	Italy	Pop	Annalisa	Annalisa Scarrone	Italian	vocals
SM23	Italy	Pop-Rock	Jovanotti	Lorenzo Cherubini	Italian	vocals
SM24	Italy	Jazz	Stefano Bollani	Stefano Bollani	Italian	piano
SM25	Italy	Metal	Damiano David	Damiano David	Italian	vocals

Table 5.5: Bands

ID Band	Country	Genre	Artist Stage Name	Formation Year	Band Name
B01	Italy	Rock	Måneskin	2016	Måneskin
B02	Italy	Pop	Negramaro	2001	Negramaro
B03	Italy	Electronic	Subsonica	1996	Subsonica
B04	UK	Rock	Coldplay	1996	Coldplay
B05	USA	Alternative	Imagine Dragons	2008	Imagine Dragons
B06	USA	Rock	Foo Fighters	1994	Foo Fighters
B07	France	Electronic	Daft Punk	1993	Daft Punk
B08	Germany	Metal	Rammstein	1994	Rammstein
B09	Iceland	Post-Rock	Sigur Ros	1994	Sigur Ros
B10	Canada	Indie	Arcade Fire	2000	Arcade Fire
B11	UK	Indie	Arctic Monkeys	2002	Arctic Monkeys
B12	UK	Alternative	Muse	1994	Muse
B13	USA	Rock	Paramore	2004	Paramore
B14	USA	Rock	Kings of Leon	1999	Kings of Leon
B15	Italy	Indie	Thegiornalisti	2009	Thegiornalisti
B16	Italy	Jazz	Fabrizio Bosso Quartet	2010	Fabrizio Bosso Quartet
B17	Italy	Metal	Lacuna Coil	1994	Lacuna Coil
B18	Italy	Pop	Boomdabash	2003	Boomdabash
B19	Italy	Folk	Modena City Ramblers	1991	Modena City Ramblers
B20	UK	Pop	The 1975	2002	The 1975
B21	USA	Pop	Maroon 5	2001	Maroon 5
B22	France	Pop	Phoenix	1999	Phoenix
B23	Italy	Jazz	Paolo Conte Trio	2015	Paolo Conte Trio
B24	Italy	Rock	Afterhours	1986	Afterhours
B25	Italy	Pop	Pinguini Tattici Nucleari	2010	Pinguini Tattici Nucleari

Relation with Performances

The *Performance_Solo_Musician* and *Performance_Band* tables establish many-to-many relationships between artists (solo and band) and the 60 performances. The dataset includes cases with single artists or bands, collaborations between two, and events with three participants.

Table 5.6: Mapping of performances to solo musicians

ID Performance	ID Solo Musician
P001	SM03
P002	SM01
P003	SM12
P004	SM08
P005	SM18
P006	SM22
P007	SM27
P008	SM07
P009	SM19
P010	SM04
P011	SM18
P012	SM09
P013	SM36
P014	SM10
P015	SM25
P016	SM14
P017	SM33
P018	SM06
P019	SM03
P020	SM09
P021	SM18
P022	SM33
P023	SM08
P024	SM39
P025	SM05

Table 5.7: Mapping of performances to bands

ID Performance	ID Band
P001	B16
P002	B02
P003	B04
P004	B15
P005	B07

ID Performance	ID Band
P006	B25
P007	B23
P008	B19
P009	B26
P010	B06
P011	B34
P012	B21
P013	B43
P014	B10
P015	B11
P016	B20
P017	B26
P018	B02
P019	B16
P020	B18
P021	B01
P022	B05
P023	B09
P024	B12
P025	B08

Stands

Table 5.8: The Stand table contains 48 entries, representing the various food, drink, and merchandise stands located at each festival (from 0 to 4 per event). Each stand is associated with a specific festival and covers a variety of well-known sponsor brands such as Coca-Cola, Heineken, Vans, and Red Bull.

ID Stand	Stand Name	Sponsor	ID Festival
ST001	Coca-Cola Drink Point	Coca-Cola	1
ST002	Burger House	McDonald's	1
ST003	Vinyl Merch Corner	Spotify	1
ST004	Heineken Beer Bar	Heineken	2
ST005	Street Pizza	Eataly	2
ST006	Roma Merch	Vans	2
ST007	Red Bull Energy Spot	Red Bull	3
ST008	Veg & Go	Veg&Go	3
ST009	Coffee & More	Lavazza	4
ST010	Panini Express	Autogrill	4
ST011	Festival Merch Napoli	Hard Rock Cafe	4
ST012	Gelato Point	Algida	5
ST013	Vans Merch Store	Vans	6
ST014	Food Station Bologna	Old Wild West	6
ST015	Drink Station Bologna	Monster Energy	6

ID Stand	Stand Name	Sponsor	ID Festival
ST016	Indie Merch Verona	Spotify	7
ST017	Beer Street Verona	Heineken	7
ST018	Sweet Bites Verona	Lindt	7
ST019	Lounge Bar Genova	Campari	8
ST020	Sicilian Street Food	Eataly	9
ST021	Fresh Drinks Bari	Coca-Cola	9
ST022	Sunset Merch Palermo	Levi's	10
ST023	Island Food Palermo	Burger King	10
ST024	Tropical Drinks Palermo	Pepsi	10
ST025	Wave Drink Bar	Red Bull	11

Products

The database models three categories of products sold at the festivals:

- 10 food items (e.g., burgers, wraps, fries, pizza),

- 10 drinks (soft drinks, beer, cocktails, water),

- 6 merchandise products (t-shirts, caps, vinyls, posters, hoodies, bottles).

Since the assignment did not require tracking product quantities, a physical inventory was not modeled. Instead, the database represents only the types of products available, each identified by a representative barcode shared among items of the same type.

Table 5.9: List of available food products with prices, expiration dates, and types.

Barcode (Food)	Name	Price (€)	Expiration Date	Type
8001234567890	Classic Burger	9.50	2026-06-30	meat
8012345678901	Vegan Wrap	8.00	2026-07-15	vegan
8023456789012	Cheese Fries	6.00	2026-06-01	snack
8034567890123	Pizza Slice	5.50	2026-06-05	vegetarian
8045678901234	Hotdog	6.50	2026-08-10	meat
8056789012345	Vegetarian Sandwich	7.00	2026-07-20	vegetarian
8067890123456	Chicken Nuggets	7.50	2026-08-01	meat
8078901234567	Nutella Crepe	5.00	2026-06-25	snack
8089012345678	Sicilian Arancino	5.50	2026-09-01	meat
8090123456789	Caesar Salad	7.00	2026-06-18	vegetarian

Table 5.10: List of available drink products with prices and expiration dates.

Barcode (Drink)	Name	Price (€)	Expiration Date
5449000000996	Coca-Cola Zero 330ml	3.00	2026-12-31

Barcode (Drink)	Name	Price (€)	Expiration Date
8712000055998	Heineken Beer 500ml	5.00	2026-12-31
9012345678901	Red Bull Energy Drink	4.50	2026-09-30
8001234000001	Natural Water 500ml	2.00	2026-11-15
8021234000002	Orange Juice	3.50	2026-10-01
8031234000003	Iced Coffee	3.00	2026-09-15
8041234000004	Gin Tonic	6.00	2026-08-20
8051234000005	Campari Spritz	6.50	2026-09-10
5000123432100	Pepsi 330ml	3.00	2026-11-01
8076543210009	Lemon Tea	2.50	2026-08-30

Table 5.11: List of merchandise products available for sale with their prices.

Barcode (Merchandise)	Name	Price (€)
2000001111111	Festival T-Shirt	25.00
2000002222222	Festival Cap	15.00
2000003333333	Poster Limited Edition	20.00
2000004444444	Vinyl Record	40.00
2000005555555	Festival Hoodie	45.00
2000006666666	Reusable Bottle	18.00

Product Sales Relations

The **Sell_Food**, **Sell_Drink**, and **Sell_Merchandise** tables record which stands sell which products. Each relation includes an **ID_Sell** as a unique identifier, linking a stand to the corresponding product barcode. This structure allows every stand to sell multiple products, and multiple stands to offer the same product type, ensuring flexibility and consistency in representing sales across the different festivals.

Table 5.12: List of food products sold at each stand, including their unique identifiers.

ID Sell	ID Stand	Barcode (Food)
SF01	ST002	8001234567890
SF02	ST002	8023456789012
SF03	ST005	8034567890123
SF04	ST005	8090123456789
SF05	ST008	8012345678901
SF06	ST010	8045678901234
SF07	ST014	8056789012345
SF08	ST020	8089012345678
SF09	ST026	8078901234567
SF10	ST034	8001234567890
SF11	ST041	8067890123456

ID Sell	ID Stand	Barcode (Food)
SF12	ST045	8023456789012

Table 5.13: List of drink products sold at each stand, including their unique identifiers.

ID Sell	ID Stand	Barcode (Drink)
SD01	ST001	5449000000996
SD02	ST004	8712000055998
SD03	ST007	9012345678901
SD04	ST009	8031234000003
SD05	ST015	8051234000005
SD06	ST017	8712000055998
SD07	ST021	5449000000996
SD08	ST025	9012345678901
SD09	ST030	8712000055998
SD10	ST035	5000123432100
SD11	ST036	9012345678901
SD12	ST042	5000123432100
SD13	ST044	8712000055998
SD14	ST047	8041234000004

Table 5.14: List of merchandise products sold at each stand, including their unique identifiers.

ID Sell	ID Stand	Barcode (Merchandise)
SM01	ST003	2000001111111
SM02	ST006	2000002222222
SM03	ST011	2000003333333
SM04	ST013	2000005555555
SM05	ST016	2000001111111
SM06	ST022	2000004444444
SM07	ST027	2000001111111
SM08	ST033	2000006666666
SM09	ST037	2000001111111
SM10	ST040	2000003333333
SM11	ST043	2000002222222
SM12	ST046	2000004444444
SM13	ST048	2000005555555

Customer

Table 5.15: The Customer entity includes 80 distinct individuals, each described by name, e-mail address, and phone number. These customers represent the audience purchasing tickets and products during the festivals.

ID Customer	Full Name	Email Address	Phone Number
C001	Luca Rossi	luca.rossi@email.com	+39 3311111101
C002	Giulia Bianchi	giulia.bianchi@email.com	+39 3311111102
C003	Marco Verdi	marco.verdi@email.com	+39 3311111103
C004	Sara Neri	sara.neri@email.com	+39 3311111104
C005	Francesco Esposito	francesco.esposito@email.com	+39 3311111105
C006	Martina Gallo	martina.gallo@email.com	+39 3311111106
C007	Davide Romano	davide.romano@email.com	+39 3311111107
C008	Guendalina Nardi	guendalina.nardi@email.com	+39 3311111108
C009	Andrea Conti	andrea.conti@email.com	+39 3311111109
C010	Sofia Varotto	sofia.varotto@email.com	+39 3311111110
C011	Simone Ricci	simone.ricci@email.com	+39 3311111111
C012	Valentina Ferri	valentina.ferri@email.com	+39 3311111112
C013	Federico Colombo	federico.colombo@email.com	+39 3311111113
C014	Alessia De Luca	alessia.deluca@email.com	+39 3311111114
C015	Matteo Barbieri	matteo.barbieri@email.com	+39 3311111115
C016	Giorgia Marchetti	giorgia.marchetti@email.com	+39 3311111116
C017	Lorenzo Testa	lorenzo.testa@email.com	+39 3311111117
C018	Camilla Costa	camilla.costa@email.com	+39 3311111118
C019	Alessandro Bruno	alessandro.bruno@email.com	+39 3311111119
C020	Silvia Rizzi	silvia.rizzi@email.com	+39 3311111120
C021	Riccardo Greco	riccardo.greco@email.com	+39 3311111121
C022	Eleonora Caruso	eleonora.caruso@email.com	+39 3311111122
C023	Tommaso Gatti	tommaso.gatti@email.com	+39 3311111123
C024	Beatrice Leone	beatrice.leone@email.com	+39 3311111124
C025	Daniele Serra	daniele.serra@email.com	+39 3311111125

Tickets

Table 5.16: A total of 150 tickets were generated and linked to 25 festivals, categorized into three types: *DAY_PASS*, *REGULAR*, and *VIP*. Prices range from €19 to €399, reflecting realistic differentiation across ticket tiers.

ID Ticket	Ticket Type	Price (€)	Start Validity	End Validity	ID Festival
T001	DAY_PASS	49.00	2025-06-20	2025-06-20	1
T002	DAY_PASS	29.00	2025-06-20	2025-06-20	1
T003	DAY_PASS	49.00	2025-06-20	2025-06-20	1
T004	DAY_PASS	39.00	2025-06-21	2025-06-21	1
T005	DAY_PASS	49.00	2025-06-20	2025-06-20	1
T006	DAY_PASS	49.00	2025-06-20	2025-06-20	1
T007	DAY_PASS	59.00	2025-07-10	2025-07-10	2

T008	DAY_PASS	49.00	2025-07-10	2025-07-10	2
T009	DAY_PASS	49.00	2025-07-10	2025-07-10	2
T010	DAY_PASS	19.00	2025-07-12	2025-07-12	2
T011	DAY_PASS	49.00	2025-07-10	2025-07-10	2
T012	DAY_PASS	49.00	2025-07-10	2025-07-10	2
T013	DAY_PASS	49.00	2025-08-05	2025-08-05	3
T014	DAY_PASS	49.00	2025-08-05	2025-08-05	3
T015	DAY_PASS	49.00	2025-08-05	2025-08-05	3
T016	DAY_PASS	49.00	2025-08-06	2025-08-06	3
T017	DAY_PASS	49.00	2025-08-05	2025-08-05	3
T018	DAY_PASS	49.00	2025-08-05	2025-08-05	3
T019	DAY_PASS	49.00	2025-09-01	2025-09-01	4
T020	DAY_PASS	49.00	2025-09-01	2025-09-01	4
T021	DAY_PASS	49.00	2025-09-01	2025-09-01	4
T022	DAY_PASS	49.00	2025-09-02	2025-09-02	4
T023	DAY_PASS	59.00	2025-09-01	2025-09-01	4
T024	DAY_PASS	49.00	2025-09-01	2025-09-01	4
T025	DAY_PASS	49.00	2026-06-14	2026-06-14	5

Ticket_Payment

Design Assumptions for Tickets

The database assumes the existence of a predefined inventory of tickets already printed and available for sale. Each ticket is uniquely identified by the primary key **ID_Ticket**.

From a relational perspective, the connection between customers and tickets follows the cardinalities (1, N) from *Customer* to *Ticket*, meaning that every customer can purchase one or multiple tickets, and (0, 1) from *Ticket* to *Customer*, allowing for unsold tickets.

To correctly represent this interaction, the associative table **Ticket_Payment** connects each sold ticket with the corresponding customer, including the attributes **Payment_Date** and **Payment_Method**.

This structure supports various purchasing patterns, from single purchases to multiple ticket acquisitions across different festivals. Overall, 120 of the 150 tickets are marked as sold, distributed across the three ticket categories.

Table 5.17

Ticket ID	Customer ID	Payment Date	Payment Method
T001	C001	2025-06-20	credit_card

Ticket ID	Customer ID	Payment Date	Payment Method
T004	C002	2025-06-21	paypal
T009	C003	2025-07-10	debit_card
T013	C004	2025-08-05	cash
T016	C005	2025-08-06	credit_card
T020	C006	2025-09-01	paypal
T022	C007	2025-09-02	debit_card
T027	C008	2026-06-14	credit_card
T031	C009	2026-07-02	cash
T035	C010	2026-07-02	paypal
T037	C011	2026-08-10	credit_card
T039	C012	2026-08-11	debit_card
T043	C013	2026-09-05	cash
T046	C014	2026-09-05	paypal
T049	C015	2027-06-22	credit_card
T052	C016	2027-06-23	paypal
T055	C017	2027-07-13	debit_card
T058	C018	2027-07-14	cash
T061	C019	2027-08-18	credit_card
T064	C020	2027-08-19	paypal
T067	C021	2027-09-10	debit_card
T070	C022	2027-09-11	cash
T073	C023	2028-06-12	credit_card
T075	C024	2028-06-12	paypal
T079	C025	2028-07-01	debit_card

Purchase Relations

The **Buy_Food**, **Buy_Drinks**, and **Buy_Merchandise** tables record which *customers purchase which products*. Each relation includes an **ID_Buy** as a unique identifier, linking a customer to the corresponding product barcode.

This structure allows every customer to **buy multiple products** (even of the same type), while each product type can be **purchased by multiple customers**, maintaining flexibility and clarity in representing purchase activities across the festivals.

The dataset includes a variety of examples where different customers buy **food items, drinks, and merchandise**, showing realistic consumer behavior within the festival environment.

Table 5.18: List of food purchases made by customers, including product barcodes.

ID Buy	Customer ID	Barcode (Food)
BF01	C026	8001234567890
BF02	C002	8034567890123

ID Buy	Customer ID	Barcode (Food)
BF03	C003	8012345678901
BF04	C004	8045678901234
BF05	C005	8056789012345
BF06	C006	8078901234567
BF07	C007	8089012345678
BF08	C008	8023456789012
BF09	C009	8067890123456
BF10	C010	8090123456789
BF11	C011	8001234567890
BF12	C012	8034567890123
BF13	C013	8012345678901
BF14	C014	8078901234567
BF15	C015	8089012345678
BF16	C016	8023456789012
BF17	C017	8045678901234
BF18	C018	8056789012345
BF19	C019	8067890123456
BF20	C020	8090123456789
BF21	C021	8001234567890
BF22	C022	8012345678901
BF23	C023	8078901234567
BF24	C024	8089012345678
BF25	C025	8023456789012

Table 5.19: List of drink purchases made by customers, including product barcodes.

ID Buy	Customer ID	Barcode (Drink)
BD01	C026	5449000000996
BD02	C027	8712000055998
BD03	C028	9012345678901
BD04	C029	8001234000001
BD05	C010	8021234000002
BD06	C031	8031234000003
BD07	C032	8041234000004
BD08	C033	8051234000005
BD09	C034	5000123432100
BD10	C035	8076543210009
BD11	C036	5449000000996
BD12	C037	8712000055998
BD13	C038	9012345678901
BD14	C039	8001234000001
BD15	C040	8021234000002
BD16	C041	8031234000003
BD17	C042	8041234000004
BD18	C008	8051234000005

ID Buy	Customer ID	Barcode (Drink)
BD19	C044	5000123432100
BD20	C045	8076543210009
BD21	C046	5449000000996
BD22	C047	8712000055998
BD23	C048	9012345678901
BD24	C049	8001234000001
BD25	C050	8021234000002

Table 5.20: List of merchandise purchases made by customers, including product barcodes.

ID Buy	Customer ID	Barcode (Merchandise)
BM01	C056	2000001111111
BM02	C057	2000002222222
BM03	C058	2000003333333
BM04	C033	2000004444444
BM05	C034	2000005555555
BM06	C061	2000006666666
BM07	C062	2000001111111
BM08	C063	2000002222222
BM09	C064	2000003333333
BM10	C065	2000004444444
BM11	C066	2000005555555
BM12	C067	2000006666666
BM13	C068	2000001111111
BM14	C069	2000002222222
BM15	C070	2000003333333
BM16	C071	2000004444444
BM17	C072	2000005555555
BM18	C073	2000006666666

Summary

In summary, the **Euphoric Events** database provides a complete and coherent representation of music festival operations in Italy, covering both **organizational** and **commercial** aspects.

It integrates entities such as **festivals**, **stages**, **performances**, **artists**, **customers**, **tickets**, **stands**, and **products**, maintaining referential integrity and realistic relationships between them.

The dataset includes **25 festivals**, **60 performances**, **100 artists** (50 solo and 50 bands), **48 stands** for 3 categories of products, **26 products**, **80 customers**, and **150 tickets** (of which 120 are sold).

Each table and relationship was designed to reflect real-world dynamics, such as artist collaborations, ticket sales, and customer purchases, while keeping the structure normalized and easy to query.

Although the data volume was limited for clarity, the model remains scalable and suitable for analytical queries, reporting, and future extensions.

Chapter 6

SQL - Queries

6.1 Introduction

This chapter illustrates the analytical phase of the project, in which a series of SQL queries were executed on the populated database in order to validate its consistency and extract meaningful insights.

The objective is to demonstrate how the designed data model supports practical information retrieval and decision-making. The queries cover a wide range of operations, including selection, filtering, grouping, and nested subqueries — to investigate different aspects of the dataset, such as ticket pricing strategies, festival characteristics, customer behavior, and artist participation.

Each query is presented with its SQL statement, the resulting output table, and a concise interpretation of the results.

Building upon the database structure introduced in the previous chapter, this analytical section serves as a bridge between design and application, testing the reliability and completeness of the implemented schema.

Through this approach, the queries not only validate the logical integrity of the system but also demonstrate its ability to generate business-relevant insights, highlighting the flexibility and analytical potential of the **"Euphoric Events"** database.

6.2 Query 1: WHERE

6.2.1 ES.1: *Find all tickets above 100€*

Query:

```
SELECT ID_Ticket, Ticket_Type, Price
FROM Ticket
WHERE Price > 100;
```

ID Ticket	Ticket Type	Price (€)
T121	REGULAR	129
T122	VIP	399
T123	REGULAR	129
T124	VIP	180
T125	REGULAR	129
T126	VIP	199
T127	REGULAR	129
T128	VIP	199
T129	REGULAR	179
T130	VIP	199
T131	REGULAR	129
T132	VIP	199
T133	REGULAR	129
T134	VIP	300
T135	REGULAR	129
T136	VIP	199
T137	REGULAR	129
T138	VIP	199
T139	REGULAR	129
T140	VIP	199
T141	REGULAR	140
T142	VIP	199
T143	REGULAR	129
T144	VIP	199
T145	REGULAR	129
T146	VIP	210
T147	REGULAR	129
T148	VIP	199
T149	REGULAR	129
T150	VIP	155

Query description: The query extracts all tickets with a price higher than 100€, returning both Regular and VIP types. The resulting dataset shows consistent price patterns, with multiple tickets priced at €129 and €199, and a maximum price of €399. This distribution highlights a structured ticketing policy, where VIP passes

clearly dominate the higher range. It indicates a tiered pricing system designed to accommodate different audience segments while maintaining balanced accessibility.

6.2.2 ES.2: *Find all festivals happening in 2027*

Query:

```
SELECT Name, Location, Start_Date, End_Date
FROM Festival
WHERE Start_Date BETWEEN '2027-01-01' AND '2027-12-31';
```

Name	Location	Start Date	End Date
Bari Music Garden	Bari	2027-06-22	2027-06-23
Palermo Sunset Fest	Palermo	2027-07-13	2027-07-15
Cagliari Wave	Cagliari	2027-08-18	2027-08-20
Padova Indie Days	Padova	2027-09-10	2027-09-11
Pisa Sound Arena	Pisa	2027-06-15	2027-06-17
Aosta Alpine Jam	Aosta	2027-07-14	2027-07-14
Pescara Festival del Mare	Pescara	2027-08-09	2027-08-10

Query description: This query lists all festivals scheduled for the year 2027, identifying seven events distributed across Italy. Cities such as Bari, Palermo, and Cagliari represent the southern area, while Aosta and Padova reflect northern coverage. The concentration of dates between June and August suggests that most festivals are organized during the summer period, reinforcing the seasonal nature of outdoor music events in the country.

6.3 Query 2: WHERE, LIMIT, LIKE

6.3.1 ES.1: *First 10 customers with email containing "gmail"*

Query:

```
SELECT Full_Name, Email_Address
FROM Customer
WHERE Email_Address LIKE '%mail%'
LIMIT 10;
```

Full Name	Email Address
Luca Rossi	luca.rossi@email.com
Giulia Bianchi	giulia.bianchi@email.com
Marco Verdi	marco.verdi@email.com
Sara Neri	sara.neri@email.com
Francesco Esposito	francesco.esposito@email.com
Martina Gallo	martina.gallo@email.com
Davide Romano	davide.romano@email.com
Guendalina Nardi	guendalina.nardi@email.com
Andrea Conti	andrea.conti@email.com
Sofia Varotto	sofia.varotto@email.com

Query description: The query filters the Customer table to display the first ten users whose e-mail address includes "mail". The output shows entirely Italian names, suggesting a homogeneous domestic user base. The dominance of the email domain could reveal that most customers prefer mainstream personal accounts rather than corporate ones, which supports a consumer-oriented rather than professional target audience.

6.3.2 ES.2: *Italian artists whose stage name starts with 'M'*

Query:

```
SELECT Artist_Stage_Name, Genre, Country
FROM Artist_Solo_Musician
WHERE Country = 'Italy' AND Artist_Stage_Name LIKE 'M%';
```

Artist Stage Name	Genre	Country
Marco Mengoni	Pop	Italy
Mara Sattei	Soul	Italy
Mario Biondi	Jazz	Italy
Mario Venuti	Funk	Italy
Madame	Pop	Italy
Manuel Agnelli	Rock	Italy
Mannarino	Folk	Italy

Query description: This query extracts all Italian artists whose stage name begins with the letter "M", displaying their musical genre and country of origin. The goal

is to retrieve a specific subset of artists based on an alphabetical criterion, allowing a focused view of some personal and musical information stored in the database.

6.3.3 ES.3: *3 cheapest VIP tickets*

Query:

```
SELECT ID_Ticket, Price, ID_Festival
FROM Ticket
WHERE Ticket_Type = 'VIP'
ORDER BY Price ASC
LIMIT 3;
```

ID Ticket	Price (€)	ID Festival
T150	155	15
T124	180	2
T126	199	3

Query description: The query isolates the three least expensive VIP tickets by using the `ORDER BY Price ASC` clause and the `LIMIT 3` operator. The output shows that their prices range between €155 and €199, and that they correspond to festivals with IDs 15, 2, and 3.

6.4 Query 3: WHERE, IN, Nested Query

6.4.1 ES.1: *Customers who bought tickets for "Roma Indie Fest"*

Query:

```
SELECT c.Full_Name, c.ID_Customer
FROM Customer c
WHERE c.ID_Customer IN (
SELECT tp.ID_Customer
FROM Ticket_Payment tp
JOIN Ticket t ON tp.Ticket_ID = t.ID_Ticket
```

```
JOIN Festival f ON t.ID_Festival = f.ID_Festival
WHERE f.Name = 'Roma Indie Fest'
);
```

Full Name	Customer ID
Marco Verdi	C003
Alessia De Luca	C014
Stefano Morelli	C029
Giorgio Leone	C041
Edoardo Ferri	C056
Elisa Orlando	C071
Greta Conti	C079

Query description: The query identifies seven customers who purchased tickets for *Roma Indie Fest*. It retrieves their names and customer IDs, providing a direct view of the attendees associated with that specific festival. The output simply lists the matching customers, allowing verification of ticket purchases related to the selected event.

6.4.2 ES.2: *VIP tickets from festivals held in 2028*

Query:

```
SELECT ID_Ticket, Price, ID_Festival
FROM Ticket
WHERE ID_Festival IN (
SELECT ID_Festival FROM Festival
WHERE Start_Date BETWEEN '2028-01-01' AND '2028-12-31'
)
AND Ticket_Type = 'VIP';
```

ID Ticket	Price (€)	ID Festival
T146	210	13
T148	199	14
T150	155	15

Query description: This query retrieves all VIP tickets associated with festivals held in 2028. The result returns three records, each showing the ticket ID, price, and

corresponding festival ID. The retrieved prices range from €155 to €210.

6.4.3 ES.3: *Customers who bought both Food and Drinks, with product names*

Query:

```
SELECT DISTINCT
c.Full_Name AS Customer,
pd.Name AS Drink_Name,
pf.Name AS Food_Name
FROM Customer c
JOIN Buy_Drinks bd ON c.ID_Customer = bd.ID_Customer
JOIN Product_Drink pd ON bd.Barcode_Drink = pd.Barcode_Drink
JOIN Buy_Food bf ON c.ID_Customer = bf.ID_Customer
JOIN Product_Food pf ON bf.Barcode_Food = pf.Barcode_Food
WHERE c.ID_Customer IN (
SELECT ID_Customer FROM Buy_Food
);
```

Customer	Drink Name	Food Name
Sofia Fabbri	Coca-Cola Zero 330ml	Classic Burger
Sofia Varotto	Orange Juice	Caesar Salad
Guendalina Nardi	Campari Spritz	Cheese Fries
Guendalina Nardi	Gin Tonic	Cheese Fries

Query description: The query lists customers who purchased both food and beverage products during the festivals. Three customers satisfy the condition—Sofia Fabbri, Sofia Varotto, and Guendalina Nardi—with one customer appearing twice due to multiple drink purchases. Combinations such as *Coca-Cola Zero 330ml + Classic Burger* or *Gin Tonic + Cheese Fries* suggest typical pairings between fast food and drinks.

6.5 Query 4: GROUP BY, 1 JOIN, AS

6.5.1 ES.1: *Average ticket price per festival*

Query:

```
SELECT f.Name AS Festival, ROUND(AVG(t.Price), 2) AS Avg_Price
FROM Festival f
JOIN Ticket t ON f.ID_Festival = t.ID_Festival
GROUP BY f.Name;
```

Festival	Average Price (€)
Ancona Pop Nights	45.67
Bari Music Garden	75.25
Bologna Groove Fest	75.25
Cagliari Wave	75.38
Firenze Summer Jam	77.75
Genova Sea Sounds	79.00
Lecce Live Energy	81.63
Milano Soundwave	99.00
Modena Metal Storm	44.17
Napoli Beats	79.00
Padova Indie Days	77.75
Palermo Sunset Fest	75.25
Parma Classic Festival	69.75
Perugia Jazz Fusion	47.33
Pisa Sound Arena	45.67
Rimini Beach Beats	46.00
Roma Indie Fest	72.88
Torino Vibes	84.00
Trieste Electro Sound	77.75
Verona Rock Nights	89.13

Query description: The query calculates the average ticket price for each festival. Prices range from approximately €44 to €99, with *Milano Soundwave* showing the highest average and *Modena Metal Storm* among the lowest.

6.5.2 ES.2: *Number of stands per festival*

Query:

```
SELECT f.Name AS Festival, COUNT(s.ID_Stand) AS Total_Stands
FROM Festival f
JOIN Stand s ON f.ID_Festival = s.ID_Festival
GROUP BY f.Name;
```

Festival	Total Stands
Ancona Pop Nights	2
Aosta Alpine Jam	1
Bari Music Garden	2
Bologna Groove Fest	3
Cagliari Wave	1
Como Lakeside Fest	2
Firenze Summer Jam	2
Genova Sea Sounds	1
Lecce Live Energy	2
Lucca Music Days	1
Milano Soundwave	3
Modena Metal Storm	2
Napoli Beats	3
Padova Indie Days	2
Palermo Sunset Fest	3
Parma Classic Festival	1
Perugia Jazz Fusion	2
Pescara Festival del Mare	2
Pisa Sound Arena	1
Reggio Vibes	2
Rimini Beach Beats	2
Roma Indie Fest	3
Torino Vibes	1
Trieste Electro Sound	1
Verona Rock Nights	3

Query description: This query counts the total number of stands available at each festival. Results vary between one and three, with most large festivals such as *Milano Soundwave* and *Roma Indie Fest* hosting three stands. The variation suggests that larger events allocate more commercial space for food, drinks, and merchandise, while smaller festivals tend to prioritize simplicity and streamlined logistics.

6.5.3 ES.3: *Number of performances per stage*

Query:

```
SELECT s.Name AS Stage, COUNT(p.ID_Performance) AS Performances
FROM Stage s
JOIN Performance p ON s.ID_Stage = p.ID_Stage
GROUP BY s.Name;
```

Stage	Performances
Acoustic Garden Firenze	3
Acoustic Stage Verona	3
Ancona Indie Beach	1
Ancona Pop Arena	1
Aosta Alpine Stage	2
Bari Acoustic Bay	1
Bari Green Garden	2
Bologna Groove Stage	1
Cagliari Wave Stage	1
Como Electro Dock	1
Como Lakeside Arena	1
Electronic Dome Roma	2
Florence Jazz Hall	1
Funk Garden Bologna	1
Genova Sea Front	1
Indie Arena Roma	1
Indie Garden Milano	1
Indie Zone Verona	2
Island Stage Palermo	1
Lecce Energy Stage	2
Lucca Green Stage	2
Main Stage Milano	1
Modena Metal Dome	2
Napoli Sunset Area	1
Napoli Underground Dome	1
Padova Indie Stage	1
Palermo Sunset Arena	1
Parma Classic Hall	2
Perugia Jazz Lounge	2
Pescara Mare Arena	1
Pisa Sound Plaza	1
Reggio Groove Hall	1
Reggio Jazz Point	2
Rimini Acoustic Spot	1
Rimini Beach Main	1
Rimini Electro Bay	1
Rock Park Roma	1
Sunset Arena Milano	2
Torino Electro Hall	2
Torino Main Square	2
Trieste Electro Dome	2
Verona Rock Arena	1

Query description: The query reports how many performances were scheduled on each stage. Most stages host between one and three shows; a few (e.g., *Acoustic*

Garden Firenze, *Acoustic Stage Verona*) reach the upper end of this range. This distribution suggests a broad allocation of performances across locations, supporting diversity in scheduling and audience flow management.

6.6 Query 5: WHERE, GROUP BY

6.6.1 ES.1: *Count food products costing more than 7€*

Query:

```
SELECT Type, COUNT(*) AS Count
FROM Product_Food
WHERE Price > 7
GROUP BY Type;
```

Type	Count
meat	2
vegan	1

Query description: The query groups food products by category and counts those priced above €7. Results show two meat-based items and one vegan option, indicating that higher prices primarily apply to meat dishes. The inclusion of a vegan alternative in this price range suggests an effort to offer premium options that cater to diverse dietary preferences.

6.6.2 ES.2: *Number of artists per genre (Italian only)*

Query:

```
SELECT Genre, COUNT(*) AS Total_Artists
FROM Artist_Solo_Musician
WHERE Country = 'Italy'
GROUP BY Genre;
```

Genre	Total Artists
Classical	2
Folk	3
Funk	1
Indie	2
Jazz	5
Metal	1
Pop	11
Pop-Rock	1
Rock	2
Soul	1

Query description: This query groups Italian artists by musical genre, showing that *Pop* dominates with 11 performers, followed by *Jazz* with five and *Folk* with three. Less frequent genres such as *Funk*, *Soul*, and *Pop-Rock* appear only once. These results highlight a balanced yet Pop-oriented music scene, with Italy's mainstream and melodic traditions remaining the most represented categories.

6.6.3 ES.3: *Count tickets by type for festivals in 2026*

Query:

```
SELECT Ticket_Type, COUNT(*) AS Num_Tickets
FROM Ticket
WHERE Start_Validity BETWEEN '2026-01-01' AND '2026-12-31'
GROUP BY Ticket_Type;
```

Ticket Type	Number of Tickets
DAY_PASS	30
REGULAR	4
VIP	4

Query description: The query counts tickets by type for all festivals scheduled in 2026. The *DAY_PASS* category dominates with 30 entries, while both *REGULAR* and *VIP* have four each. This pattern indicates that short-term attendance is significantly more popular, reflecting a general trend among festival-goers who prefer single-day experiences over extended or premium access options.

6.7 Query 6: GROUP BY, HAVING, AS

6.7.1 ES.1: *Festivals with more than 2 stands*

Query:

```
SELECT f.Name AS Festival, COUNT(s.ID_Stand) AS Num_Stands
FROM Festival f
JOIN Stand s ON f.ID_Festival = s.ID_Festival
GROUP BY f.Name
HAVING COUNT(s.ID_Stand) > 2;
```

Festival	Number of Stands
Bologna Groove Fest	3
Milano Soundwave	3
Napoli Beats	3
Palermo Sunset Fest	3
Roma Indie Fest	3
Verona Rock Nights	3

Query description: The query lists all festivals that feature more than two stands. Six events meet this condition, including *Bologna Groove Fest*, *Milano Soundwave*, and *Verona Rock Nights*. These festivals typically represent larger or more commercially active events, where a higher number of stands often correlates with increased opportunities for sponsorships, product sales, and audience interaction.

6.7.2 ES.2: *Artists with more than 3 performances in ascending order*

Query:

```
SELECT a.Artist_Stage_Name AS Artist, COUNT(ps.ID_Performance) AS Num_Performances
FROM Artist_Solo_Musician a
JOIN Performance_Solo_Musician ps ON a.ID_Solo_Musician = ps.ID_Solo_Musician
GROUP BY a.Artist_Stage_Name
HAVING COUNT(ps.ID_Performance) > 3
ORDER BY Num_Performances ASC;
```

Artist	Number of Performances
Damiano David	4
Lady Gaga	4
Ludovico Einaudi	4
Francesco De Gregori	5
Mara Sattei	5
Mario Biondi	5
Purple Disco Machine	5
Andrea Bocelli	6
Laura Pausini	6
Paolo Fresu	7
Marco Mengoni	8
David Guetta	10

Query description: The query identifies artists performing in four or more events, sorted by ascending number of appearances. Results highlight prominent figures such as *David Guetta* with ten performances and *Marco Mengoni* with eight, indicating both international and domestic artists with significant recurring participation. This consistency demonstrates their popularity and sustained demand across multiple festivals.

6.7.3 ES.3: *Drink types with average price > 4€*

Query:

```
SELECT Name AS Drink_Name, AVG(Price) AS Avg_Price
FROM Product_Drink
GROUP BY Name
HAVING AVG(Price) > 4;
```

Drink Name	Average Price (€)
Campari Spritz	6.5
Gin Tonic	6.0
Heineken Beer 500ml	5.0
Red Bull Energy Drink	4.5

Query description: The query calculates the average price per drink type, returning only those with averages exceeding €4. Products such as *Campari Spritz*, *Gin Tonic*, and *Heineken Beer* show mid-range values between €4.5 and €6.5, indicating that alcoholic beverages generally occupy a higher price segment. In contrast, soft drinks

tend to remain below this threshold, mirroring real-world pricing and consumption trends.

6.8 Query 7: WHERE, GROUP BY, HAVING, AS

6.8.1 ES.1: *Festivals in 2027 with average ticket price > 70€*

Query:

```
SELECT f.Name AS Festival, AVG(t.Price) AS Avg_Price
FROM Festival f
JOIN Ticket t ON f.ID_Festival = t.ID_Festival
WHERE f.Start_Date BETWEEN '2027-01-01' AND '2027-12-31'
GROUP BY f.Name
HAVING AVG(t.Price) > 70;
```

Festival	Average Price (€)
Bari Music Garden	75.25
Cagliari Wave	75.38
Padova Indie Days	77.75
Palermo Sunset Fest	75.25

Query description: This query filters all festivals in 2027 with an average ticket price exceeding €70. Four events meet this condition — *Bari Music Garden*, *Cagliari Wave*, *Padova Indie Days*, and *Palermo Sunset Fest*. These festivals likely attract larger audiences and more prominent performers, which justifies their higher pricing tiers. The results also suggest that mid-sized Italian cities can sustain premium-priced events with consistent demand.

6.8.2 ES.2: *Customers who bought more than 2 drinks*

Query:

```
SELECT ID_Customer, COUNT(*) AS Drinks_Bought
FROM Buy_Drinks
WHERE Barcode_Drink IS NOT NULL
GROUP BY ID_Customer
HAVING COUNT(*) > 2;
```

Customer ID	Drinks Bought
C037	3

Query description: The query identifies customers who purchased more than two drinks. Only one customer (*C037*) meets this condition, with a total of three purchases. This limited occurrence indicates that most festival-goers typically make one or two drink purchases, reflecting casual, open-air event consumption patterns rather than heavy or repeated buying behavior.

6.9 Query 8: WHERE, Nested Query, GROUP BY

6.9.1 ES.1: *Customers who bought tickets with an average price above the global average*

Query:

```
SELECT c.Full_Name AS Customer, ROUND(AVG(t.Price), 2) AS Avg_Ticket_Price
FROM Customer c
JOIN Ticket_Payment tp ON c.ID_Customer = tp.ID_Customer
JOIN Ticket t ON tp.Ticket_ID = t.ID_Ticket
WHERE t.Price > (
SELECT AVG(Price) FROM Ticket
)
GROUP BY c.Full_Name
ORDER BY Avg_Ticket_Price DESC
LIMIT 10;
```

Customer	Average Ticket Price (€)
Pietro Rizzi	399
Matilde Galli	399
Lorenzo Fonti	399
Ilaria Gentile	399
Federico Colombo	399
Veronica Romano	300
Sara Neri	300
Beatrice Rinaldi	300
Alessandro Bruno	300
Sofia Varotto	210

Query description: The query compares each customer's average ticket price to the overall mean, returning only those whose spending exceeds it. Ten individuals qualify, with the highest-paying group purchasing tickets up to €399. This subset of high-spending customers likely includes frequent VIP or premium attendees, highlighting a valuable market segment for future promotional targeting and exclusive experiences.

6.9.2 ES.2: *Most popular musical genres among festivals held after 2026*

Query:

```
SELECT p.Genre AS Music_Genre,
COUNT(p.ID_Performance) AS Total_Performances
FROM Performance p
WHERE p.ID_Stage IN (
SELECT s.ID_Stage
FROM Stage s
JOIN Festival f ON s.ID_Festival = f.ID_Festival
WHERE f.Start_Date >= '2026-01-01'
)
GROUP BY p.Genre
ORDER BY Total_Performances DESC
LIMIT 5;
```

Music Genre	Total Performances
Electronic	10
Pop	9
Classical	6

Music Genre	Total Performances
Jazz	4
Indie	3

Query description: The query ranks musical genres by total performances in festivals taking place after 2026. *Electronic* music leads with ten performances, followed by *Pop* and *Classical*. These results reflect a noticeable shift towards modern and electronic genres in post-2026 events, signaling evolving audience preferences and changes in festival programming strategies.

6.10 Query 9: WHERE, GROUP BY, HAVING, 1 JOIN

6.10.1 ES.1: *Top 5 festivals with the highest average ticket price*

Query:

```
SELECT f.Name AS Festival,
ROUND(AVG(t.Price), 2) AS Avg_Ticket_Price,
COUNT(t.ID_Ticket) AS Total_Tickets
FROM Festival f
JOIN Ticket t ON f.ID_Festival = t.ID_Festival
WHERE t.Price > 50
GROUP BY f.Name
HAVING AVG(t.Price) > (
SELECT AVG(Price) FROM Ticket
)
ORDER BY Avg_Ticket_Price DESC
LIMIT 5;
```

Festival	Average Ticket Price (€)	Total Tickets
Milano Soundwave	264	2
Verona Rock Nights	214.5	2
Torino Vibes	189	2
Cagliari Wave	169.5	2
Trieste Electro Sound	164	2

Query description: The query lists the five festivals with the highest average ticket prices, led by *Milano Soundwave* (€264) and *Verona Rock Nights* (€214.5). The consistently elevated averages across these events suggest that urban locations and major headliners justify premium pricing. It also emphasizes a clear relationship between city reputation, festival prestige, and ticket cost structure.

6.10.2 ES.2: *Artists performing in more than one festival*

Query:

```
SELECT a.Artist_Stage_Name AS Artist,
COUNT(DISTINCT f.ID_Festival) AS Festivals_Played
FROM Artist_Solo_Musician a
JOIN Performance_Solo_Musician ps ON a.ID_Solo_Musician = ps.ID_Solo_Musician
JOIN Performance p ON ps.ID_Performance = p.ID_Performance
JOIN Stage s ON p.ID_Stage = s.ID_Stage
JOIN Festival f ON s.ID_Festival = f.ID_Festival
WHERE a.Country = 'Italy'
GROUP BY a.Artist_Stage_Name
HAVING COUNT(DISTINCT f.ID_Festival) > 1
ORDER BY Festivals_Played DESC
LIMIT 10;
```

Artist	Festivals Played
Paolo Fresu	6
Marco Mengoni	6
Mara Sattei	5
Laura Pausini	5
Mario Biondi	4
Ludovico Einaudi	4
Andrea Bocelli	4
Levante	3
Damiano David	3
Ultimo	2

Query description: The query lists artists who performed at more than one festival. *Paolo Fresu* and *Marco Mengoni* lead with six festivals each, followed by *Laura Pausini* and *Mara Sattei* with five. This repeated participation highlights the continued popularity of top artists, illustrating how well-known performers sustain audience engagement and strengthen the Italian festival circuit's visibility and market appeal.

6.11 Query 10:WHERE, GROUP BY, HAVING, 2 JOINs

6.11.1 ES.1: *Festivals where customers bought the most merchandise items*

Query:

```
SELECT f.Name AS Festival,
COUNT(bm.ID_Buy) AS Total_Merch_Sold
FROM Festival f
JOIN Stand s ON f.ID_Festival = s.ID_Festival
JOIN Sell_Merchandise sm ON s.ID_Stand = sm.ID_Stand
JOIN Product_Merchandise pm ON sm.Barcode_Merchandise = pm.Barcode_Merchandise
JOIN Buy_Merchandise bm ON pm.Barcode_Merchandise = bm.Barcode_Merchandise
WHERE pm.Price > 10
GROUP BY f.Name
HAVING COUNT(bm.ID_Buy) > 2
ORDER BY Total_Merch_Sold DESC
LIMIT 5;
```

Festival	Total Merchandise Sold
Verona Rock Nights	3
Roma Indie Fest	3
Rimini Beach Beats	3
Pescara Festival del Mare	3
Perugia Jazz Fusion	3

Query description: The query identifies the festivals with the highest number of merchandise items sold, focusing on products priced above €10. Five festivals reach the same top value of three items each — including *Verona Rock Nights*, *Roma Indie Fest*, and *Pescara Festival del Mare*. The even distribution across multiple festivals highlights widespread consumer interest in merchandise, indicating that purchases are not concentrated within a single genre or location but rather represent a broad engagement trend.

6.11.2 ES.2: *Artist (solo and band musician) with highest average ticket price (with regular or VIP ticket pass) across multiple performances*

Query:

```
SELECT Artist,
ROUND(AVG(Price), 2) AS Avg_Ticket_Price,
COUNT(DISTINCT ID_Performance) AS Performances_Analyzed
FROM (
SELECT a.Artist_Stage_Name AS Artist, p.ID_Performance, t.Price
FROM Artist_Solo_Musician a
JOIN Performance_Solo_Musician ps ON a.ID_Solo_Musician = ps.ID_Solo_Musician
JOIN Performance p ON ps.ID_Performance = p.ID_Performance
JOIN Stage s ON p.ID_Stage = s.ID_Stage
JOIN Festival f ON s.ID_Festival = f.ID_Festival
JOIN Ticket t ON f.ID_Festival = t.ID_Festival
WHERE t.Ticket_Type IN ('VIP','REGULAR')
UNION ALL
SELECT b.Artist_Stage_Name AS Artist, p.ID_Performance, t.Price
FROM Band b
JOIN Performance_Band pb ON b.ID_Band = pb.ID_Band
JOIN Performance p ON pb.ID_Performance = p.ID_Performance
JOIN Stage s ON p.ID_Stage = s.ID_Stage
JOIN Festival f ON s.ID_Festival = f.ID_Festival
JOIN Ticket t ON f.ID_Festival = t.ID_Festival
WHERE t.Ticket_Type IN ('VIP','REGULAR')
)
GROUP BY Artist
HAVING COUNT(DISTINCT ID_Performance) > 1
ORDER BY Avg_Ticket_Price DESC
LIMIT 10;
```

Artist	Average Ticket Price (€)	Performances Analyzed
Negramaro	230.67	3
Arlo Parks	214.50	2
Pinguini Tattici Nucleari	199.25	4
The 1975	197.67	3
Marco Mengoni	197.33	3
Paolo Fresu	192.88	4
Purple Disco Machine	190.63	4
Planet Funk	184.20	5
Fabrizio Bosso Quartet	182.70	5
Thegiornalisti	177.67	3

Query description: The query identifies both solo and band artists with the highest average ticket prices based on regular or VIP passes across multiple performances.

Negramaro ranks first with an average of €230.67, followed by *Arlo Parks* and *Pinguini Tattici Nucleari*. These findings highlight how top Italian and international acts maintain strong ticket valuations, confirming their consistent commercial appeal, recurrent audience engagement, and sustained influence across various festivals.

6.12 Final Thoughts

Overall, the set of SQL queries presented in this chapter provided an overview of the database's structure and the key relationships among entities. Through the use of conditional filters, grouping operations, and nested queries, it was possible to extract meaningful insights on ticket pricing strategies, customer purchasing behavior, and the commercial scale of different events.

Chapter 7

Conclusion

The Euphoric Events project achieved all the required objectives of database design and implementation. Starting from the textual problem statement, we analyzed the scenario and identified the main entities, relationships, and constraints necessary to represent the organization of music festivals in Italy.

Through the conceptual and logical design phases, we built a normalized and consistent model that ensures data integrity and scalability. The SQL implementation successfully translated the ER and logical models into a functional database structure, including primary and foreign keys, integrity constraints, and realistic data inserted.

A total of around 25 to 120 tuples per table were generated to populate the database, ensuring realistic and consistent data across all entities. Multiple queries covering different SQL categories, such as filtering, grouping, nested subqueries, and joins, were implemented to demonstrate the analytical capabilities and flexibility of the system.

These queries range from simple selections using WHERE to complex nested joins and aggregations, providing meaningful insights such as artist participation, ticket distribution, and customer purchasing behavior.

Overall, the project delivers a complete, self-contained database that accurately fulfills the initial requirements. The resulting schema is clear, efficient, and easily extendable, demonstrating the full cycle of database design: from conceptual analysis to implementation and query-based analysis.